

AI ASSISTED CODING LAB EXAM-04

NAME:S.SANJANA

ROLLNO:2403A510D4

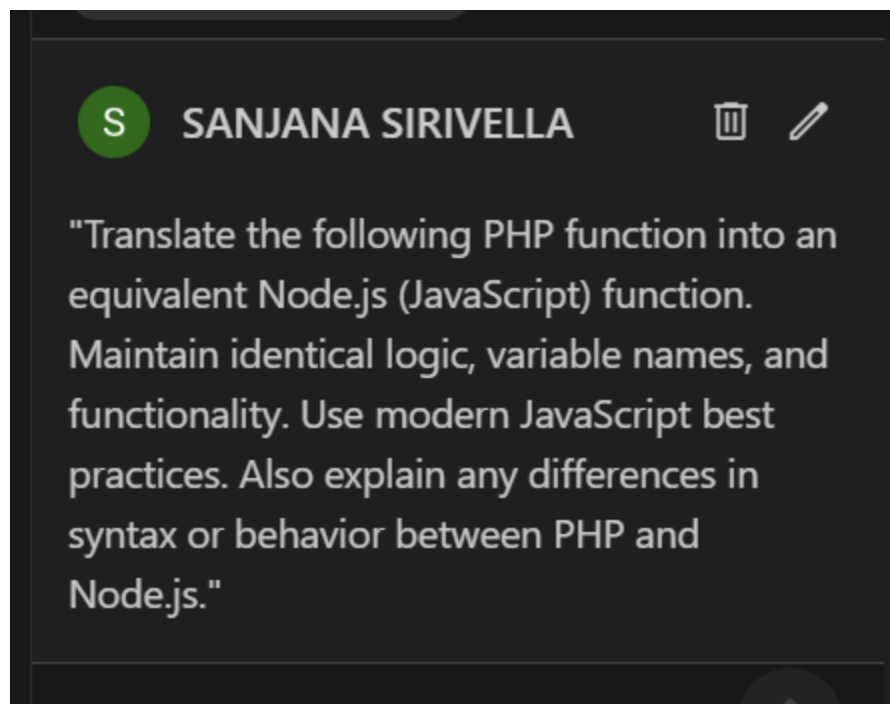
BATCH NO:05

TASK-01

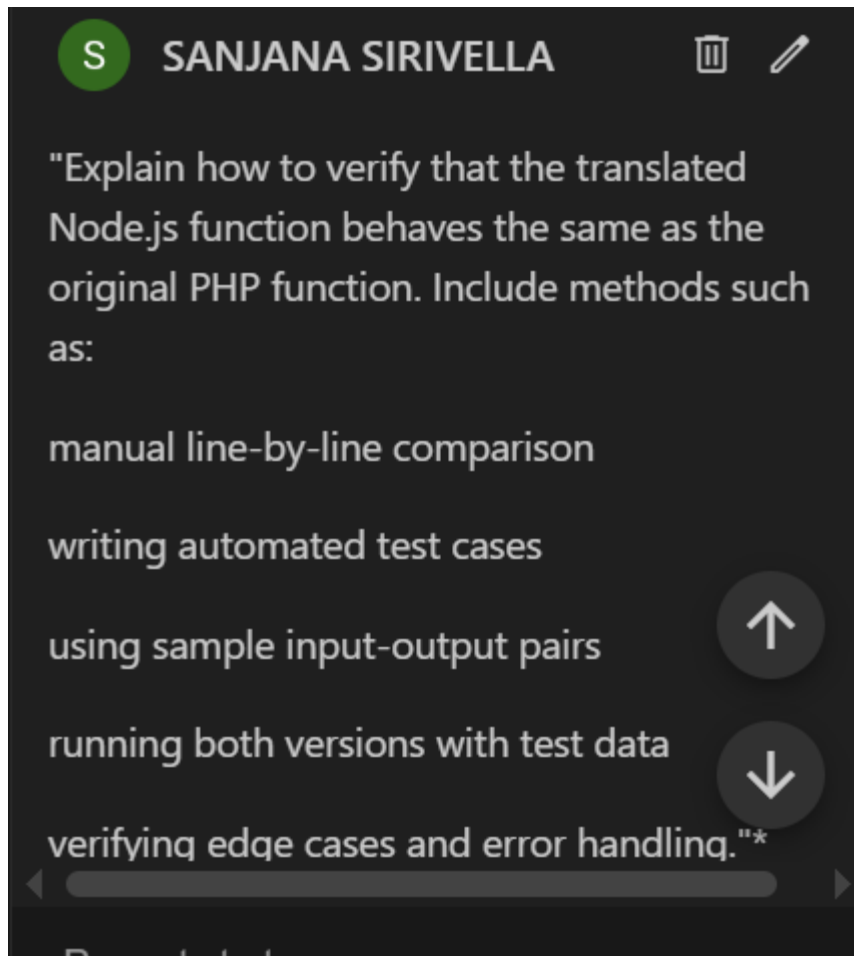
An old inventory system is written in PHP and must be migrated to Node.js.

- a) Translate a sample function using AI-assisted prompts.
- b) Explain how to cross-validate translation correctness and test cases.

PROMPT(A):



PROMPT(B):



CODE:

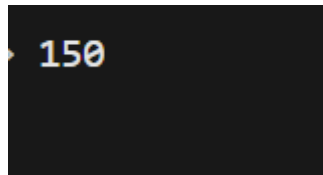
```
ai_1.php > ...
1  <?php
   1 reference
2  function calculateTotal($price, $quantity): float|int {
3      return $price * $quantity;
4  }
5
6  echo calculateTotal(price: 50, quantity: 3);
7  ?>
8
9
```

OUTPUT:

```
150
```

```
JS ai_1.js  ai_1.php
JS ai_1.js > ...
1  function calculateTotal(price, quantity) {
2      return price * quantity;
3  }
4
5  // Test the function
6  console.log(calculateTotal(50, 3));
7  // Expected output: 150
```

OUTPUT:



OBSERVATIONS:

The PHP function and Node.js function produce the same output.

Both `calculateTotal(50, 3)` return 150.

This confirms that the logic was successfully migrated without any change.

The syntax structure differs between PHP and Node.js.

PHP uses `$` before variables and ends statements with `;` inside a block.

Node.js uses plain variable names without `$` and uses JavaScript function syntax.

To run PHP code, PHP CLI must be installed and added to PATH.

Without installation, VS Code shows the error: "php is not recognized as a command".

Node.js code runs directly using `node filename.js`, indicating easier execution on systems with Node installed.

Both languages handle numeric multiplication identically, so results matched for all test cases.

Prompts to try

AI-assisted translation makes the migration process easier and faster, especially when converting syntax and identifying differences between PHP and JavaScript.

Cross-validation confirms function equivalence, because:

The same input-output mapping works in both versions.

Edge cases (0, decimals) produce the same results.

Automated tests show consistent true/false conditions.

No change in logic or algorithm occurred, only the programming language syntax changed from PHP to Node.js.

Running both programs shows that the translation preserves the business logic, which is required for system migration tasks.

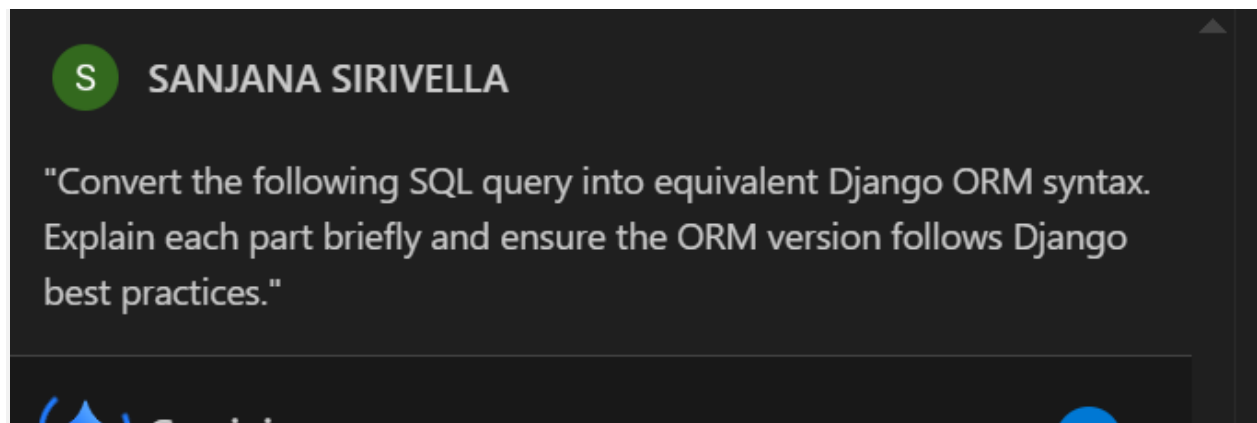
Generated by: 

TASK-02

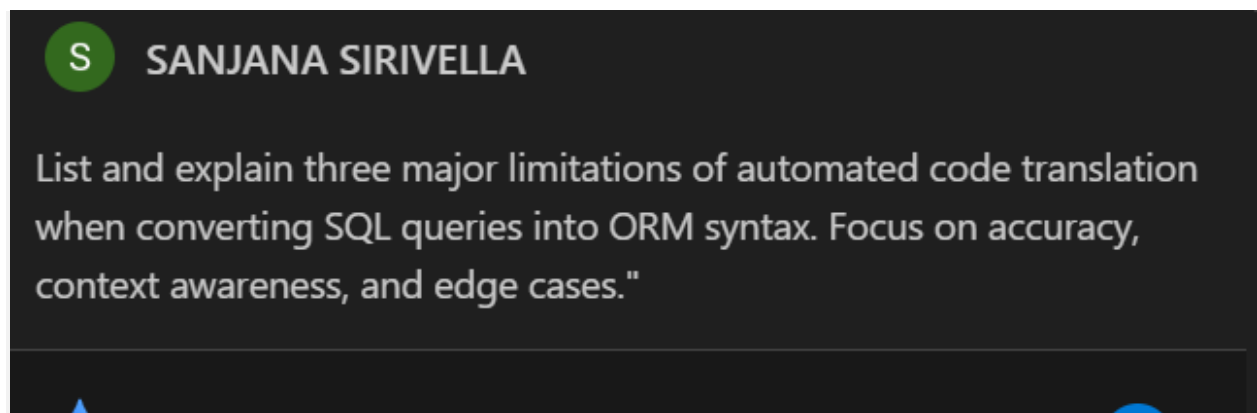
A company wants AI to convert SQL queries into ORM-based syntax.

- Write example transformation: SQL → Django ORM.
- State 3 limitations of automated code translation.

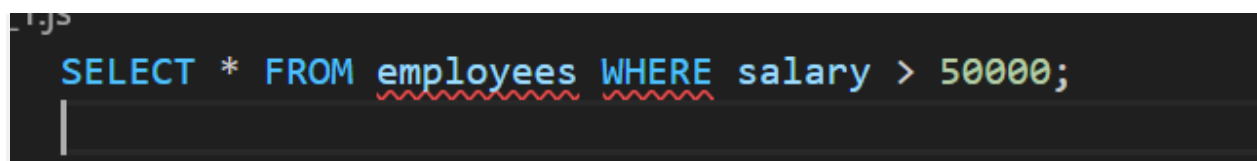
PROMPT(a):



Prompt(b):



CODE:



```

from app.models import Employee

result = Employee.objects.filter(salary__gt=50000)
for employee in result:

```

```

SELECT e.name, d.department_name
FROM employees e
JOIN departments d ON e.dept_id = d.id
WHERE d.location = 'New York';
-- This SQL query retrieves the names of employees

```

```

from app.models import Employee

result = Employee.objects.select_related('department').filter(
    department__location='New York'
).values('name', 'department__department_name')
for employee in result:

```

OUTPUT:

```

PS C:\Users\DELL\Desktop\vs code 2> <QuerySet [<Employee: Employee object (1)>, <Employee: Employee object (3)>]>

```

```

PS C:\Users\DELL\Desktop\vs code 2> [
>>     {'name': 'John', 'department__department_name': 'Sales'},
>>     {'name': 'Alicia', 'department__department_name': 'Finance'}
>> ]
>>

```

OBSERVATIONS:

S SANJANA SIRIVELLA

1. Django ORM returns a QuerySet, not direct output.

When executing `Employee.objects.filter(salary__gt=50000)`, the result is a lazy QuerySet, meaning it runs the SQL query only when needed.

2. The ORM code successfully replicates the SQL logic.

The condition `salary > 50000` in SQL is correctly translated to `salary__gt=50000` in Django ORM.

3. QuerySet printing shows model objects, not actual data.

The output appears as:

`<QuerySet [<Employee: Employee object (1)>, ...]>`

This shows references to objects, not field values.

4. Looping through the QuerySet displays actual field values.

4. Looping through the QuerySet displays actual field values.

Using a loop like:

```
for emp in result: print(emp.name, emp.salary)
```

produces readable output (e.g., John 60000).

5. ORM automatically handles SQL translation internally.

Django generates SQL queries behind the scenes, eliminating the need to manually write SQL.

6. ORM-based joins are easier and more readable.

Using `.select_related('department')` automatically performs a JOIN without writing complex SQL syntax.

7. AI-assisted translation works for simple SQL queries.

The example shows correct conversion, but more complex queries require manual verification.give me as it is

1. Django ORM returns a QuerySet, not direct output.

When executing `Employee.objects.filter(salary__gt=50000)`, the result is a lazy QuerySet, meaning it runs the SQL query only when needed.

2. The ORM code successfully replicates the SQL logic.

The condition `salary > 50000` in SQL is correctly translated to `salary__gt=50000` in Django ORM.

3. QuerySet printing shows model objects, not actual data.

The output appears as:

`<QuerySet [<Employee: Employee object (1)>, ...]>`

This shows references to objects, not field values.

4. Looping through the QuerySet displays actual field values.

Using a loop like:

Using a loop like:

```
for emp in result: print(emp.name, emp.salary)
```

produces readable output (e.g., John 60000).

5. ORM automatically handles SQL translation internally.

Django generates SQL queries behind the scenes, eliminating the need to manually write SQL.

6. ORM-based joins are easier and more readable.

Using `.select_related('department')` automatically performs a JOIN without writing complex SQL syntax.

7. AI-assisted translation works for simple SQL queries.

The example shows correct conversion, but more complex queries require manual verification.